# Assignment 5

Banking Application: Design an inheritance-based banking system with account types and overridden methods.
https://github.com/krrish-talesara/Programming-in-Java-Lab-

## 1: Account.java

```
import java.util.ArrayList;

public class Account {
    protected int accountNumber;
    protected double balance;
    protected Customer customer;
    private ArrayList<String> transactionHistory = new ArrayList<>();

    public Account(int accountNumber, double balance, Customer customer) {
        this.accountNumber = accountNumber;
        this.balance = balance;
        this.customer = customer;
    }

    public void deposit(double amount) {
        balance += amount;
        transactionHistory.add("Deposited: " + amount + ", Balance: " + balance);
        System.out.println("Deposited: " + amount);
        System.out.println("Balance: " + balance);
    }

    public void withdraw(double amount) throws InsufficientBalanceException {
        if (amount > balance)
            throw new InsufficientBalanceException("Insufficient Balance");

        balance -= amount;
        transactionHistory.add("Withdrawn: " + amount + ", Balance: " + balance);
        System.out.println("Withdrawn: " + amount);
        System.out.println("Balance: " + balance);
    }

    public void displayAccount() {
        customer.displayCustomer();
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Balance: " + balance);
    }

    public void printTransactionHistory() {
        for (String t : transactionHistory) {
            System.out.println(t);
        }
    }
}
```

## 2: Customer.java

```
public class Customer {
    public String customerID;
```

```
    public String firstName;
    public String lastName;
    public String email;
    public String phoneNumber;
    public String address;
    protected String aadharID;
    protected String panID;

    public Customer(String customerID, String firstName, String lastName,
                String email, String phoneNumber, String address,
                String aadharID, String panID) {
        this.customerID = customerID;
        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
        this.phoneNumber = phoneNumber;
        this.address = address;
        this.aadharID = aadharID;
        this.panID = panID;
    }

    public void displayCustomer() {
        System.out.println("Customer ID: " + customerID);
        System.out.println("Name: " + firstName + " " + lastName);
        System.out.println("Email: " + email);
        System.out.println("Phone: " + phoneNumber);
        System.out.println("Address: " + address);
        System.out.println("Aadhar: " + aadharID);
        System.out.println("PAN: " + panID);
    }
}
```

# 3: CurrentAccount.java

```
public class CurrentAccount extends Account {
    private double minimumBalance = 500;

    public CurrentAccount(int accountNumber, double balance, Customer customer) {
        super(accountNumber, balance, customer);
    }

    public void checkMinimumBalance() {
        if (balance < minimumBalance)
            System.out.println("Minimum balance not maintained");
        else
            System.out.println("Minimum balance maintained");
    }

    @Override
    public void withdraw(double amount) throws InsufficientBalanceException {
        if (amount > balance)
            throw new InsufficientBalanceException("Insufficient Balance");

        balance -= amount;
        System.out.println("Current Withdrawal: " + amount);
    }
}
```

# 4: SavingsAccount.java

```
public class SavingsAccount extends Account {
   private double interestRate = 0.06;

   public SavingsAccount(int accountNumber, double balance, Customer customer) {
      super(accountNumber, balance, customer);
   }

   public void calculateInterest() {
      double interest = balance * interestRate;
      System.out.println("Interest: " + interest);
   }

   @Override
   public void withdraw(double amount) throws InsufficientBalanceException {
      if (balance - amount < 500)
         throw new InsufficientBalanceException("Minimum balance 500 required");

      balance -= amount;
      System.out.println("Savings Withdrawal: " + amount);
   }
}
```

# 5: Loan.java

```
public class Loan {
   private double loanAmount;
   private int months;
   private double emi;

   public Loan(double loanAmount, int months) {
      this.loanAmount = loanAmount;
      this.months = months;
      this.emi = loanAmount / months;
   }

   public void payEMI(Account acc) throws InsufficientBalanceException {
      if (acc.balance < emi)
         throw new InsufficientBalanceException("Insufficient balance for EMI");

      acc.balance -= emi;
      System.out.println("EMI Paid: " + emi);
      System.out.println("Remaining Balance: " + acc.balance);
   }

   public void displayLoan() {
      System.out.println("Loan Amount: " + loanAmount);
      System.out.println("EMI: " + emi);
   }
}
```

# 6: Main.java

```
import java.util.ArrayList;

public class Main {
   public static void main(String[] args) {
      try {
         ArrayList<Customer> customers = new ArrayList<>();

         Customer c1 = new Customer("1", "A", "B", "a@mail.com",
```

```
            "1111111111", "City", "123", "PAN1");

        Customer c2 = new Customer("2", "C", "D", "b@mail.com",
            "2222222222", "City", "456", "PAN2");

        customers.add(c1);
        customers.add(c2);

        ArrayList<Account> accounts = new ArrayList<>();

        SavingsAccount sa1 = new SavingsAccount(1001, 50000, c1);
        CurrentAccount ca1 = new CurrentAccount(2001, 30000, c1);

        accounts.add(sa1);
        accounts.add(ca1);

        System.out.println("Transactions");

        sa1.deposit(2000);
        sa1.withdraw(1000);
        sa1.calculateInterest();

        Loan loan = new Loan(12000, 12);
        loan.displayLoan();
        loan.payEMI(sa1);

        displayInfo(customers, accounts);

      } catch (InsufficientBalanceException e) {
        System.out.println(e.getMessage());
      }
    }

    public static void displayInfo(ArrayList<Customer> customers, ArrayList<Account> accounts) {
      for (Customer cust : customers) {
        cust.displayCustomer();

        for (Account acc : accounts) {
          if (acc.customer.customerID.equals(cust.customerID)) {
            System.out.println("Account: " + acc.accountNumber);
            System.out.println("Type: " + acc.getClass().getSimpleName());
            System.out.println("Balance: " + acc.balance);
          }
        }
      }
    }
}
```

# 7: InsufficientBalanceException.java

```
public class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String msg) {
      super(msg);
    }
}
```

# 8: InvalidAccountException.java

```
public class InvalidAccountException extends Exception {
    public InvalidAccountException(String msg) {
```

```
        super(msg);
    }
}
```

**Output)**

```
Transactions
Deposited: 5000.0
Balance: 65000.0
Savings Withdrawal: 2000.0
Interest: 3780.0
Loan Amount: 24000.0
EMI: 1000.0
EMI Paid: 1000.0
Remaining Balance: 62000.0
Customer ID: 1
Name: A B
Email: a@mail.com
Phone: 1111111111
Address: City
Aadhar: 123
PAN: PAN1
Account: 1001
Type: SavingsAccount
Balance: 62000.0
Account: 2001
Type: CurrentAccount
Balance: 35000.0
Customer ID: 2
Name: C D
Email: b@mail.com
Phone: 2222222222
Address: City
Aadhar: 456
PAN: PAN2
PS C:\Users\HP\Documents\College\Sem 4\PIJ Lab\Assignment 5>
```

**Exception :**

```
Savings Withdrawal: 2000.0
Interest: 3780.0
Loan Amount: 24000.0
EMI: 1000.0
EMI Paid: 1000.0
Remaining Balance: 62000.0
Expected withdrawal exception: Minimum balance 500 required
Loan Amount: 1000000.0
EMI: 1000000.0
Expected EMI exception: Insufficient balance for EMI
```